GestorEFi — Especificación Técnica

Documento de Contexto y Directivas para el Agente AntiGravity IDE

Proyecto: Club de Ciencias
Entorno: Local / Offline
Versión: 1.0 (Especificación)

ARQUITECTURA	PERSISTENCIA	MODO DE OPERACIÓN
Desktop Nativa	**SQLite3 Local**	**PC Depósito**
Python + CustomTkinter	Cero dependencia web	100% Funcional sin Internet

1. CONTEXTO OPERATIVO Y PROBLEMA

La asignatura de Educación Física en la escuela secundaria cuenta con un depósito físico de materiales distanciado del edificio principal, zona donde la señal Wi-Fi no llega y no se dispone de cableado de red Ethernet. Actualmente, el registro de pelotas, conos, colchonetas e indumentaria se realiza de forma informal, derivando en pérdida de stock, falta de trazabilidad en los recreos y dificultad para justificar compras de reposición.

La solución requiere un software ejecutable instalado en la única computadora fija del depósito. Debe ofrecer una interfaz ágil, moderna e intuitiva que permita a los docentes consultar disponibilidad, registrar préstamos rápidos a alumnos para el recreo y auditar roturas o extravíos.

2. JUSTIFICACIÓN TECNOLÓGICA DEL STACK

Se ha seleccionado una arquitectura de escritorio liviana basada en Python frente a alternativas web o marcos pesados como Electron. Esta decisión responde directamente a los condicionantes de infraestructura y mantenibilidad:

Tecnología Evaluada	Ventajas Principales	Desventajas / Inconvenientes	Decisión
Python + CustomTkinter	Ejecución liviana, inicio instantáneo, instalación directa mediante `.exe` o script, base de datos integrada.	Limitado a la computadora donde está instalado.	**SELECCIONADO**
Electron (JS / Node)	Interfaz gráfica estilizada con HTML/CSS.	Consumo excesivo de RAM y espacio en disco para una PC de depósito.	Descartado
Tauri (Rust + Web)	Muy liviano y de alto rendimiento.	Requiere conocimientos de Rust para configurar la capa nativa.	Descartado
Aplicación Web Tradicional	Acceso multidispositivo desde celulares.	Inviable por falta de conectividad Wi-Fi/Ethernet en el depósito.	Descartado

3. ARQUITECTURA DE BASE DE DATOS (SQLITE)

El almacenamiento de datos es 100% local mediante SQLite. El agente debe generar el archivo de base de datos `gestor_efi.db` en la primera ejecución ejecutando las sentencias DDL correspondientes.

Tabla: `materiales`

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `nombre`: TEXT NOT NULL (Ej. Pelota de Básquet N°7)
- `categoria`: TEXT NOT NULL (Pelotas, Conos, Indumentaria, Colchonetas)
- `stock_total`: INTEGER NOT NULL
- `stock_disponible`: INTEGER NOT NULL
- `estado`: TEXT (Nuevo, Bueno, Gastado, Inutilizable)

Tabla: `prestamos`

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `material_id`: INTEGER (FK -> materiales.id)
- `alumno_nombre`: TEXT NOT NULL
- `curso_division`: TEXT NOT NULL
- `fecha_salida`: DATETIME DEFAULT CURRENT_TIMESTAMP
- `fecha_devolucion`: DATETIME NULL
- `estado`: TEXT (Activo, Devuelto, No Devuelto)

Tabla: `historial_bajas`

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `material_id`: INTEGER (FK -> materiales.id)
- `cantidad`: INTEGER NOT NULL
- `motivo`: TEXT NOT NULL (Rotura, Pérdida, Robado)
- `responsable_registro`: TEXT NOT NULL
- `fecha`: DATETIME DEFAULT CURRENT_TIMESTAMP

Tabla: `usuarios`

- `id`: INTEGER PRIMARY KEY AUTOINCREMENT
- `username`: TEXT UNIQUE NOT NULL
- `password_hash`: TEXT NOT NULL
- `rol`: TEXT CHECK(rol IN ('ADMIN', 'CONSULTA'))

4. CONTROL DE ACCESO Y MATRIZ DE ROLES

Rol	Credenciales de Acceso	Permisos Habilitados en la App
ADMINISTRADOR	Acceso Docente (Con Contraseña)	Carga de nuevos materiales, edición de stock, registro de bajas definitivas por rotura/pérdida, gestión de usuarios.
CONSULTA / RECREO	Acceso General (Sin clave o clave simple)	Búsqueda y visualización de stock disponible, registro rápido de préstamos y devoluciones a alumnos para el recreo.

5. FLUJOS DE TRABAJO PRINCIPALES

Préstamo Rápido en Recreos

1. El alumno solicita un material (ej. Pelota de Fútbol).
2. El usuario busca el insumo en el buscador general.
3. Se ingresa el nombre del alumno y curso.
4. El sistema descuenta 1 del `stock_disponible` y

Auditoría y Bajas de Stock

1. Un profesor detecta un elemento roto o perdido.
2. Accede con rol Administrador.
3. Selecciona el ítem y registra la baja indicando motivo (Rotura/Extravío) y responsable.

6. ESTRUCTURA DEL CÓDIGO Y DIRECTIVAS PARA ANTIGRAVITY

El agente de IA debe estructurar el repositorio con un diseño modular siguiendo el patrón MVC (Modelo-Vista-Controlador) adaptado a CustomTkinter. Todo el código generado debe ser autosuficiente, comentado en español y libre de dependencias externas no declaradas.

```
gestorefi/ ├── main.py # Punto de entrada principal ├── requirements.txt # Dependencias (customtkinter) ├── database/ │ ├── connection.py # Conexión y creación de DB SQLite │ └── queries.py # Consultas SQL preparadas ├── gui/ │ ├── app.py # Ventana principal y navegación │ ├── login_frame.py # Módulo de autenticación │ ├── inventory_frame.py # Pantalla de gestión de stock │ ├── loans_frame.py # Pantalla de préstamos de recreo │ └── audit_frame.py # Historial de bajas y reportes └── models/ ├── material.py # Clases y lógica de negocio └── loan.py
```

7. PROTOCOLO DE VIBECODING Y REGLAS PARA EL AGENTE

- **Parámetros Consultas SQL:** Usar SIEMPRE consultas preparadas con tuplas (`?`) para evitar inyecciones SQL. Nunca concatenar variables directas en strings SQL.
- **Estilo de UI (CustomTkinter):** Implementar paleta de colores profesional de dos tonos (Dark Blue `#1e3a8a` / Light Gray `#f8fafc`). Garantizar que la interfaz sea adaptable a pantallas de 1366x768 (resolución estándar de PC de escuela).
- **Manejo de Errores:** Envolver operaciones de base de datos e interfaz en bloques `try-except` capturando `sqlite3.Error` y desplegando mensajes de diálogo claros con `tkinter.messagebox`.
- **Validación de Campos:** Garantizar que las cantidades numéricas no acepten valores negativos ni texto, e impedir préstamos si el `stock_disponible` es menor a 1.

8. ROADMAP DE DESARROLLO POR PROMPTS

Fase	Entregable del Agente	Criterio de Aceptación
Fase 1	Base de datos (`database/`) y modelos.	Creación automática de `gestor_efi.db` con tablas e inserción de datos semilla.
Fase 2	Módulo de Autenticación (`login_frame.py`).	Validación correcta de roles ADMIN y CONSULTA con bloqueo de funciones.
Fase 3	Gestor de Inventario y Filtros (`inventory_frame.py`).	Tabla dinámica de insumos con filtro instantáneo por categoría y nombre.
Fase 4	Módulo de Préstamos Rápidos (`loans_frame.py`).	Registro de préstamos a alumnos con actualización automática de stock en tiempo real.
Fase 5	Módulo de Bajas e Historial (`audit_frame.py`).	Registro auditado de roturas/pérdidas y generación de reporte de resumen.